SHOPWAVE PLATFORM

Complete User Manual & Platform Architecture Guide

AI-Powered Multi-Retailer Recommendation Engine & E-Commerce Application
Cross-Shopping Amazon India, Flipkart, Croma & ShopWave

Core Stack: Flutter 3.47 • FastAPI AI Backend • Google Gemini 2.5 Flash • Supabase (PostgreSQL with RLS)

Document Version: 2.0 (Production Release)	**Release Date:** October 2026
Architecture: Microservices + Hybrid Client	**AI Engine:** Gemini 2.5 Flash (Multimodal Vision & NLP)
Test Coverage: 100% Passed (18 Flutter / 6 Backend)	**Target Platforms:** Web (Chrome), Android, iOS, Windows

Welcome to ShopWave:
This manual provides step-by-step instructions for operating the ShopWave mobile and web platform. Whether you are evaluating the intelligent multi-retailer recommendation algorithms, testing the Gemini 2.5 Flash conversational assistant, comparing prices across authorized Indian retailers, or exploring the PostgreSQL RLS security policies, this document details all procedures, endpoints, and architectural components.

1. Executive Overview & System Architecture

ShopWave aggregates live product listings from **Amazon India**, **Flipkart**, **Croma**, and local **ShopWave** merchants into a single unified shopping platform. Using Google's Gemini 2.5 Flash multimodal model and Bayesian recommendation mathematics, ShopWave provides real-time side-by-side matrices, conversational budget shopping assistance, image-based visual matching, and automated price-drop tracking with offline resiliency.

Key Architecture Components:
- **Flutter Frontend:** Clean Architecture with BLoC 9.0, get_it DI locator, and go_router deep linking.
- **FastAPI AI Orchestrator:** High-throughput async REST microservice with Uvicorn, computing Bayesian confidence ratings.
- **Google Gemini 2.5 Flash:** Multimodal NLP for intent parsing, budget extraction, and visual image categorization.
- **Supabase PostgreSQL:** Row-Level Security (RLS) tenant isolation for user profiles, carts, wishlists, and orders.
- **Offline Resiliency Layer:** Local storage caching providing zero-connectivity graceful degradation.

2. Getting Started & Environment Setup

Prerequisites: Python 3.10+, Flutter 3.27+ (Dart 3.13+), and Google Chrome or Android emulator.

Step 1: Verify Environment (.env)

Ensure the root .env file is configured with the following parameters:

```
SUPABASE_URL=https://xyzcompany.supabase.co
SUPABASE_ANNONKEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.fake_key
BACKEND_API_URL=http://localhost:8000
GEMINI_API_KEY=your_gemini_api_key_here
```

Step 2: Start the FastAPI AI Backend

From the root directory, start the Uvicorn server:

```
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

Interactive OpenAPI/Swagger documentation is available at: **http://127.0.0.1:8000/docs**.

Step 3: Launch the Flutter Web Application

Run the application targeting Google Chrome:

```
flutter run -d chrome --web-port 3000
```

The application will bundle all assets and open automatically at: **http://localhost:3000**.

3. Complete Feature Walkthrough & How to Use

Feature 1: Interactive Home Screen & Multi-Store Discovery

- **Store Filter Bar:** Tap on *All Stores*, *Amazon India*, *Flipkart*, or *Croma* pills to instantly refine product listings.
- **AI Copilot Hero Banner:** Dark obsidian gradient card with one-touch prompt chips ('Best Phone under 25k', 'Gaming Laptops') for instant AI shopping consultations.
- **Discount Ribbons & Ratings:** Clearly badges percentage savings (e.g. '15% OFF') and 4.8★ rating scores on all product cards.
- **Optimistic Wishlisting:** Tap the floating heart icon on any card to save items with offline caching.

Feature 2: Google Gemini 2.5 Flash AI Shopping Copilot

Accessible via the top glowing AI icon or hero button. The AI assistant processes conversational shopping requests:

- **Budget Parsing:** Extracts constraints such as *'under 50k'*, *'below Rs. 30,000'*, or *'worth Rs. 2,500'*.
- **Feature Detection:** Automatically identifies keywords for ANC, Battery, High-Res Camera, Gaming, or AMOLED display.
- **Transparent Explanations:** Explains exactly why a product is recommended (e.g. *'Fits budget of Rs. 50,000, saves Rs. 5,000, and has 4.8★ rating from 1,200 reviews'*).

Feature 3: Side-by-Side Product Comparison Matrix

Tap the comparison icon in the AppBar or **Compare Across Stores** on any Product Detail screen. The matrix evaluates:

- **Best Price & Savings:** Identifies the single lowest price across Amazon India, Flipkart, and Croma.
- **Confidence Scores:** Compares review volume and weighted Bayesian scores.
- **Delivery & Warranty:** Outlines delivery timelines (e.g., 2-Day Prime vs Standard) and return policies.
- **Direct Redirects:** One-touch affiliate redirection to authorized stores via url_launcher.

Feature 4: Multimodal Visual Image Search

Tap **Visual Search** to identify and price-match products from photos:

- **Camera & Gallery Input:** Take a real-time photo or select an existing image.
- **Gemini Vision Pipeline:** Extracts product categories, model clues, and color palettes from image base64 data.
- **Cross-Store Matching:** Calculates cosine similarity to match visual features against available store catalogs.

Feature 5: 30-Day Historical Price Tracking & Drop Alerts

- **Interactive Timeline:** Graph on every product detail page displaying lowest, highest, and 30-day average prices.
- **Trend Detection:** Automatically calculates whether current pricing is *'dropping'*, *'stable'*, or *'rising'*.
- **Price Drop Alerts:** Subscribe to target price thresholds to receive automated notification alerts.

Feature 6: Zero-Connectivity Offline Resiliency

Powered by OfflineCacheService and shared_preferences, ShopWave provides full offline resilience. If internet connection drops, the app immediately switches to locally cached categories, products, wishlists, and user preferences with zero blank screens or crashes.

4. Backend REST API Endpoints Specification

Endpoint	Method	Description	Sample Request Payload / Param
/health	**GET**	Health check & active providers status	None
/api/v1/search	**POST**	Aggregates products across all stores with sorting & price bounds	{"query": "laptop", "min_price": 40000}
/api/v1/assistant	**POST**	Gemini AI intent extraction & shopping advice	{"message": "best phone under 30k"}
/api/v1/compare	**POST**	Generates side-by-side merchant comparison matrix	{"product_title": "iPhone 15"}
/api/v1/visual-search	**POST**	Multimodal image attribute extraction & visual match	{"image_base64": "..."}
/api/v1/price-history/{id}	**GET**	Returns 30-day price timeline & trend direction	Path: id=prod_1

5. Database Security Architecture (PostgreSQL RLS)

All user tables implement strict Row-Level Security (RLS) in supabase/migrations/01_ai_ecommerce_enhancements.sql:

- **wishlists:** Composite primary key on (user_id, product_id) with RLS policy auth.uid() = user_id ensuring complete tenant isolation.
- **product_reviews:** Stores verified buyer reviews and star ratings (1-5).
- **price_history & price_alerts:** Automated threshold tracking and alert subscriptions.
- **user_preferences:** Default budget limits, preferred retailers, and notification flags.

6. Frequently Asked Questions & Troubleshooting

Q: What happens if there is no internet connection?
A: ShopWave features zero-connectivity offline resiliency. Curated categories and high-definition products are cached locally, and search queries fall back to verified offline data seamlessly.

Q: Can I use my own Supabase database keys?
A: Yes. Update SUPABASE_URL and SUPABASE_ANNONKEY in the root .env file. The app will immediately link to your remote PostgreSQL instance and authenticate live users.

Q: How do merchant purchase redirects work?
A: When tapping 'Buy from Amazon' or 'Buy from Flipkart', the app uses url_launcher to open the merchant portal with attached affiliate tracking identifiers (e.g. shopwave-21), ensuring proper attribution.

Q: What Gemini model is used and what is the latency?
A: The system utilizes gemini-2.5-flash via asynchronous HTTP connection. Typical response latency is between 1.2 to 2.2 seconds, with automatic deterministic fallback if a network timeout occurs.